DETECTION OF SQL INJECTION ATTACKS USING SPLUNK SIEM

INTRODUCTION

- Web applications are widely used and are often targeted by attackers. One of the most common web attacks is SQL Injection, where an attacker manipulates database queries through user input.

This project demonstrates:

- How a vulnerable web application is created
- How SQL Injection attacks are performed
- How attack logs are generated
- How Splunk SIEM detects these attacks
- Visualization of attack data using a dashboard
- Creation of alerts and configuration of real-time email notifications, along with a simple Telegram notification alert

PROJECT ENVIRONMENT SETUP

System	Purpose
Windows	Splunk Enterprise (Manager)
Ubuntu	Victim system (Web application + Forwarder)

SOFTWARE USED

- Python 3
- Flask Framework
- SQLite Database
- Splunk Enterprise

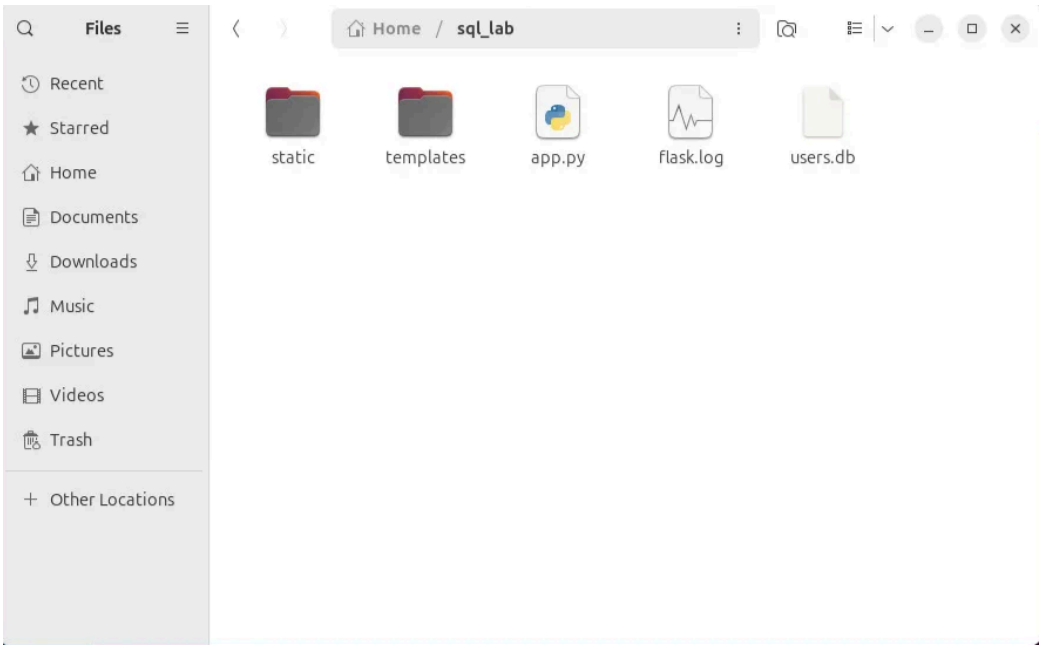
- Splunk Universal Forwarder

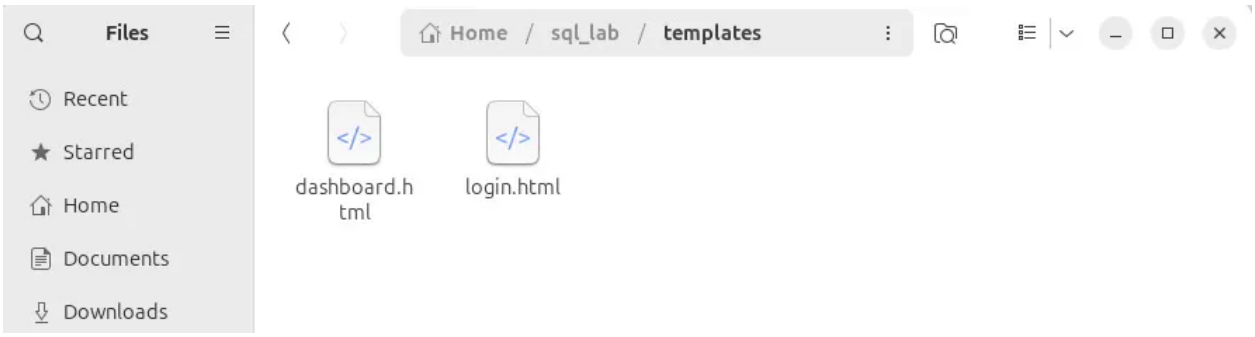
CREATION OF VULNERABLE WEB APPLICATION IN UBUNTU

This section explains exactly how the website was created.

Project Directory Structure

```
sql_lab/
├── app.py
├── users.db
├── flask.log
├── templates/
│   ├── login.html
│   └── dashboard.html
└── static/
└── style.css
```







DESCRIPTION OF EACH FILE

- app.py

- Main Flask application file
- Handles routing, database connection, and login logic
- Intentionally vulnerable to SQL Injection

- users.db

- SQLite database file
- Stores user credentials (username and password)
- Used for authentication

- flask.log

- Log file generated by the Flask application

- Stores login attempts and user input
- Used by Splunk for attack detection

🔹 templates/login.html

- Frontend login page
- Takes username and password from the user

🔹 templates/dashboard.html

- Dashboard page shown after successful login
- Indicates successful authentication

🔹 static/style.css

- CSS file for styling login and dashboard pages
- Makes the UI look professional

VULNERABILITY EXPLANATION

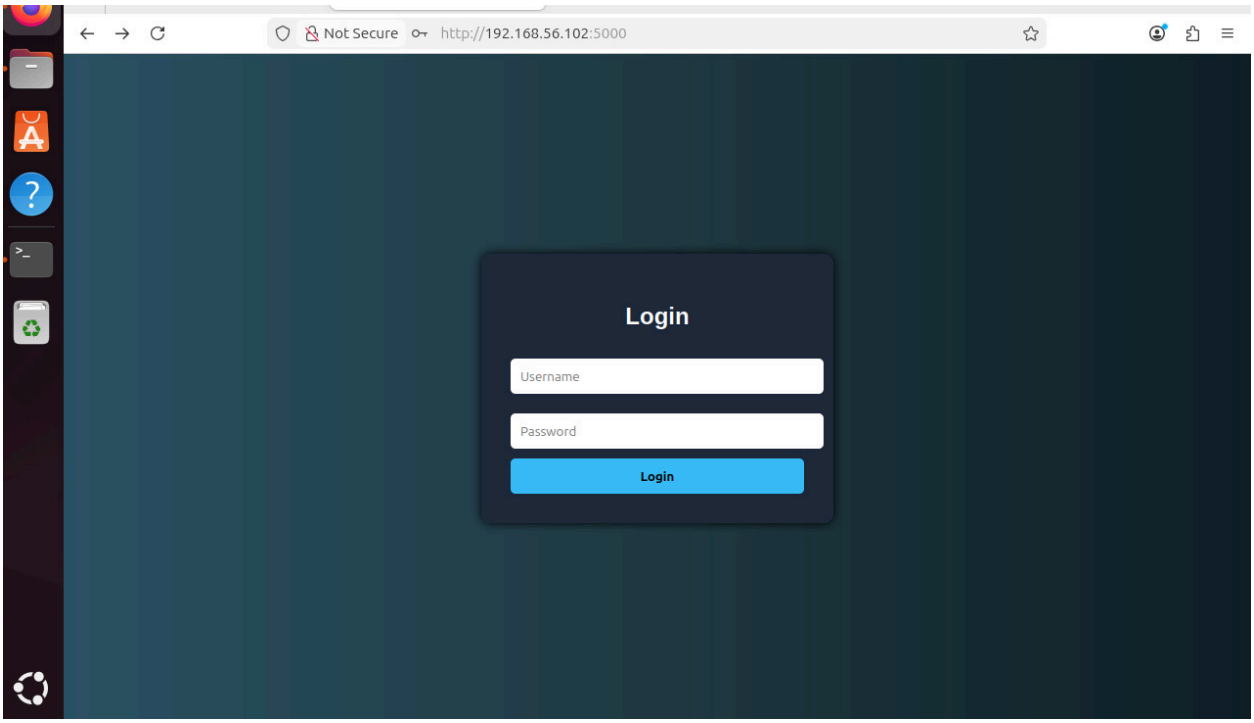
- User input is directly inserted into the SQL query
- No input validation or parameterized queries
- This makes the application vulnerable to SQL Injection

RUNNING THE WEB APPLICATION

python3 app.py

- Accessed using browser:
- http://<ubuntu-ip>:5000

- In my case it was http://192.168.56.102:5000



SQL INJECTION ATTACK EXECUTION

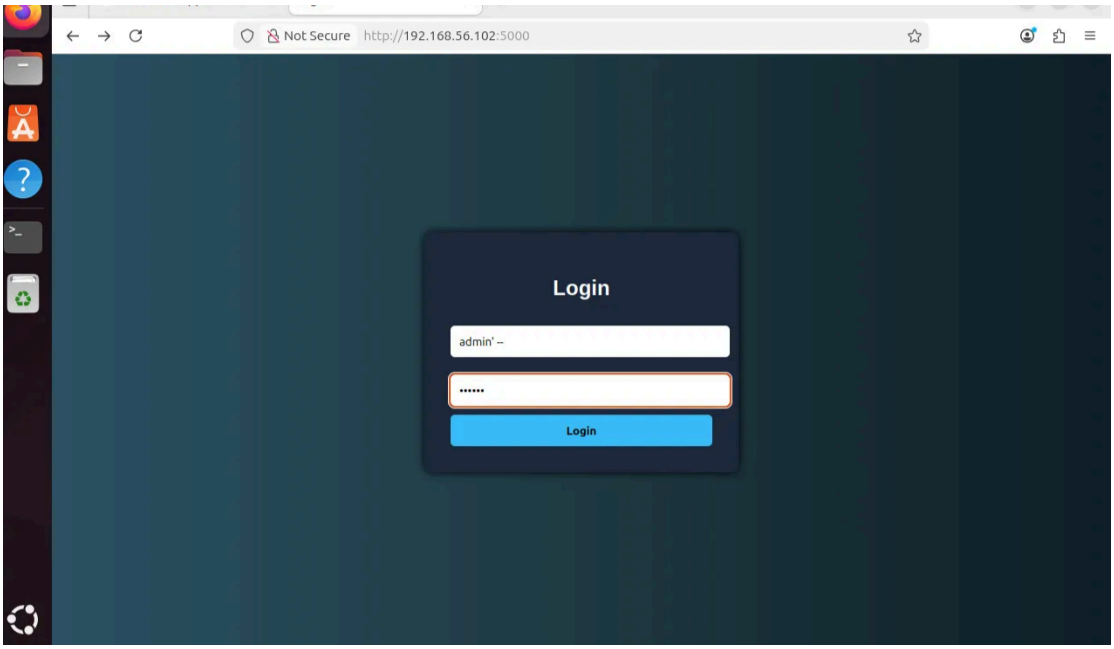
Normal Login Attempt
Username: admin
Password: admin123

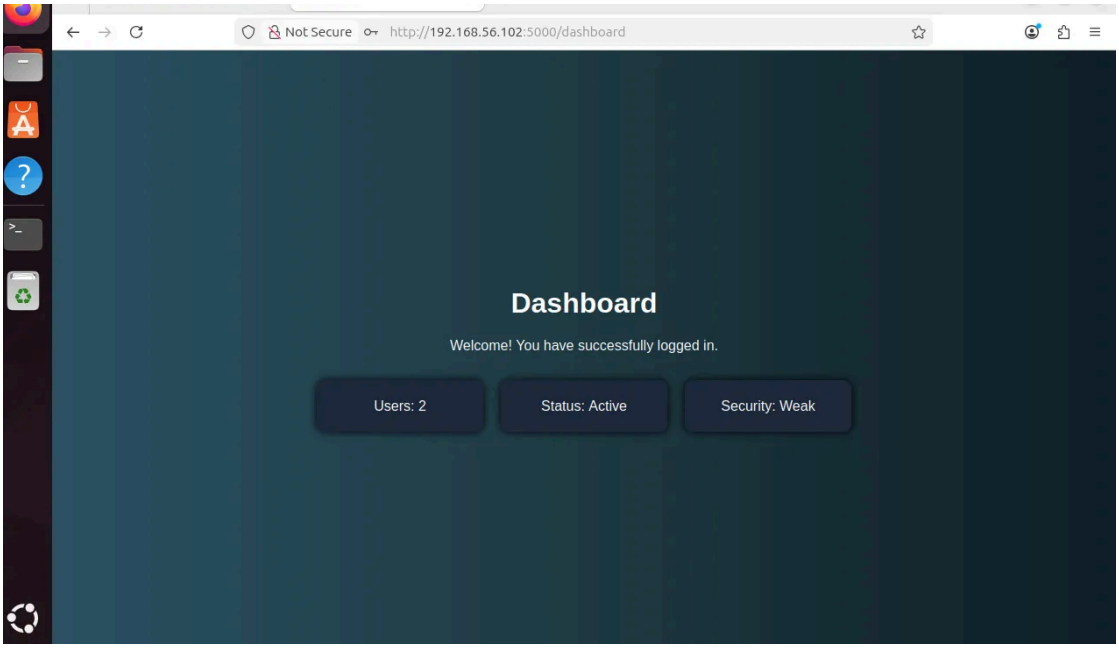
SQL INJECTION ATTACK

Username: admin' --
Password: anything

Result:

- Login bypassed
- Dashboard accessed without valid password

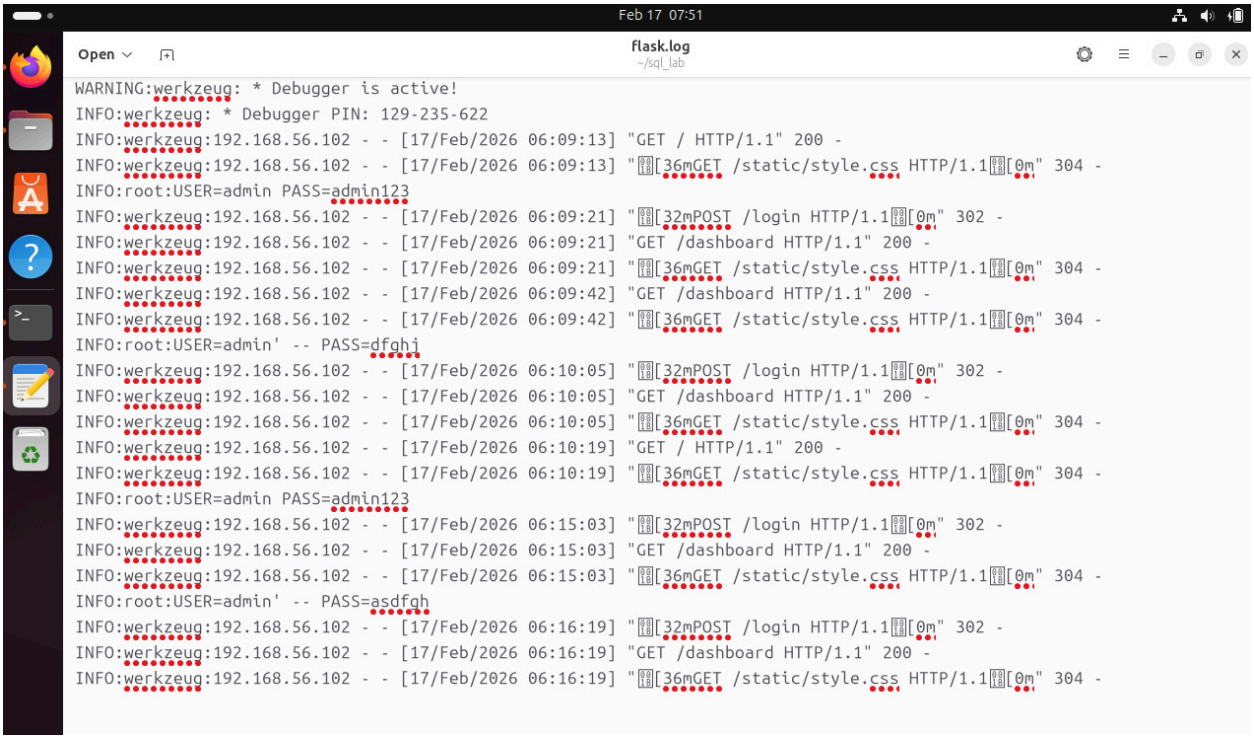




LOG GENERATION

- Each login attempt is logged in flask.log:

  USER=admin' -- PASS=anything



LOG FORWARDING TO SPLUNK

/opt/splunkforwarder/bin/splunk add monitor /home/vboxuser/sql_lab/flask.log
/opt/splunkforwarder/bin/splunk restart

DETECTION OF SQL INJECTION IN SPLUNK

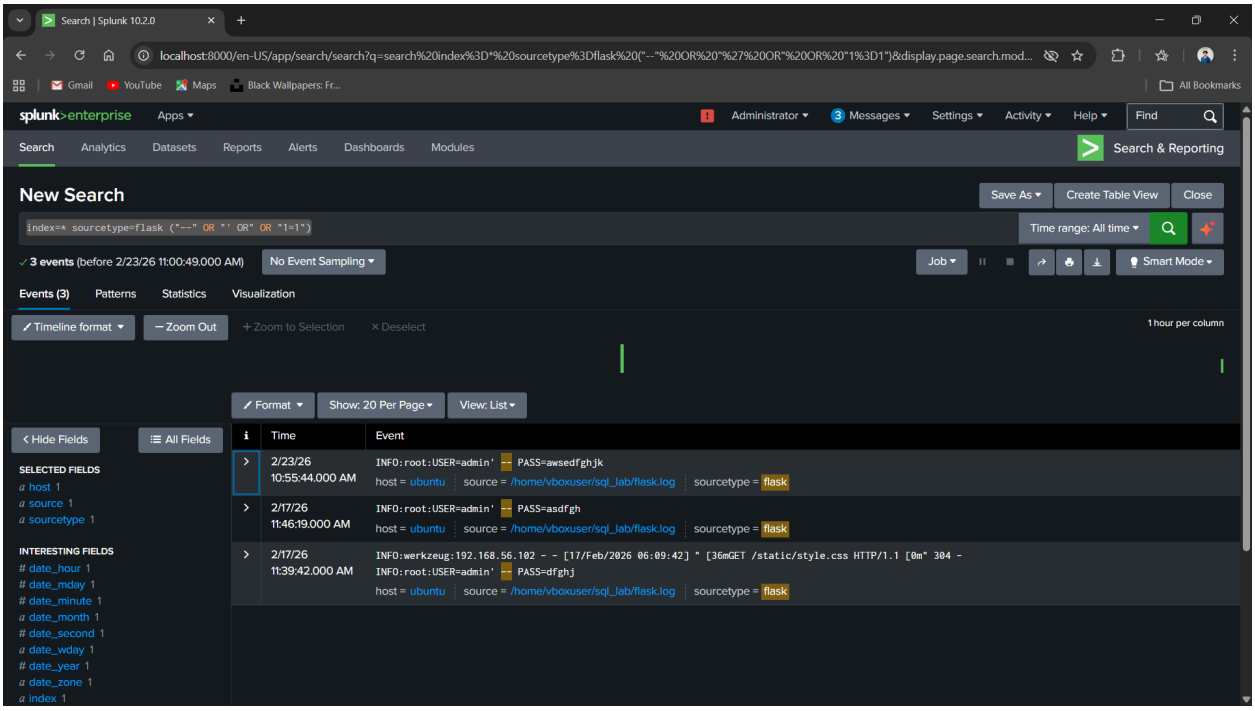
Splunk Search Query:
index=* sourcetype=flask ("--" OR "' OR" OR "1=1")

RESULTS

- SQL Injection attack detected successfully
- Attack logs visible in Splunk
- Demonstrated real-time attack monitoring

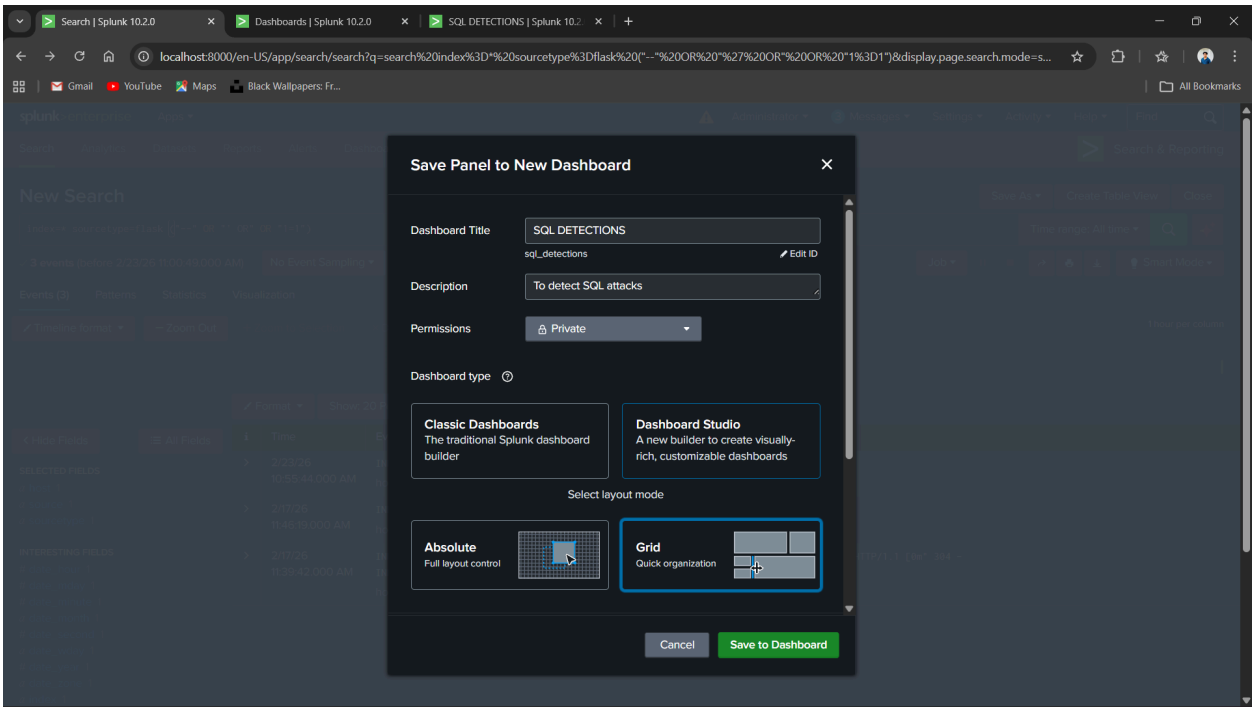
LOG CAPTURING ON SPLUNK ENTERPRISE

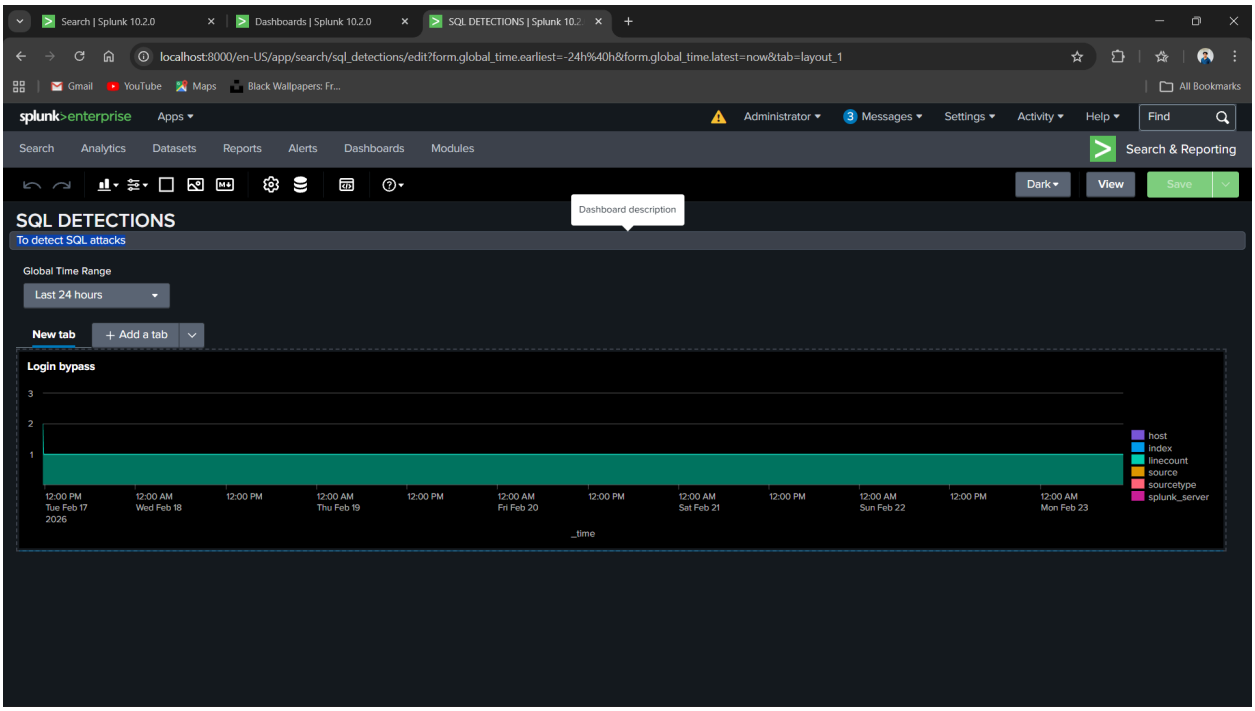
I captured the login bypass logs from the login page and forwarded it into Splunk Enterprise



CREATING AND ORGANIZING DASHBOARD

Next, I created a dashboard to visualize, monitor, and analyze data in real-time.





HOW THE ATTACK WAS IDENTIFIED

User input was monitored through application logs

Logs were forwarded to Splunk SIEM

Splunk searches identified suspicious patterns such as:

' OR

-

-

1=1

These patterns confirmed SQL Injection attempts

SPLUNK ENUMERATION QUERY :

source="flask.log" ("--" OR "' OR" OR "1=1")

This step helps enumerate malicious inputs and attacker behavior.

ROOT CAUSE OF SQL INJECTION

```
db = sqlite3.connect("users.db")
q = f"SELECT * FROM users WHERE username='{u}' AND password='{p}'"
r = db.execute(q).fetchone()
db.close()
```

Why the attack worked:

q = f"SELECT * FROM users WHERE username='{u}' AND password='{p}'"

MITIGATION FOR THIS ATTACK

```
db = sqlite3.connect("users.db")
q = "SELECT * FROM users WHERE username=? AND password=?"
r = db.execute(q, (u, p)).fetchone()
db.close()
```

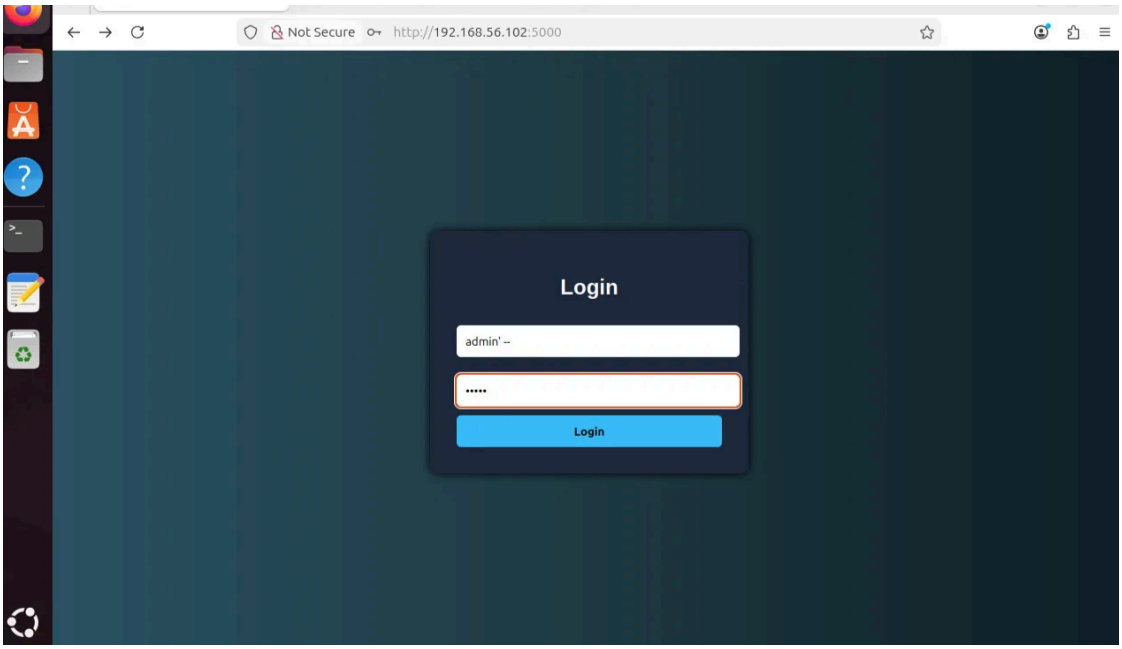
Vulnerable Code:

```
q = f"SELECT * FROM users WHERE username='{u}' AND password='{p}'"
r = db.execute(q).fetchone()
```

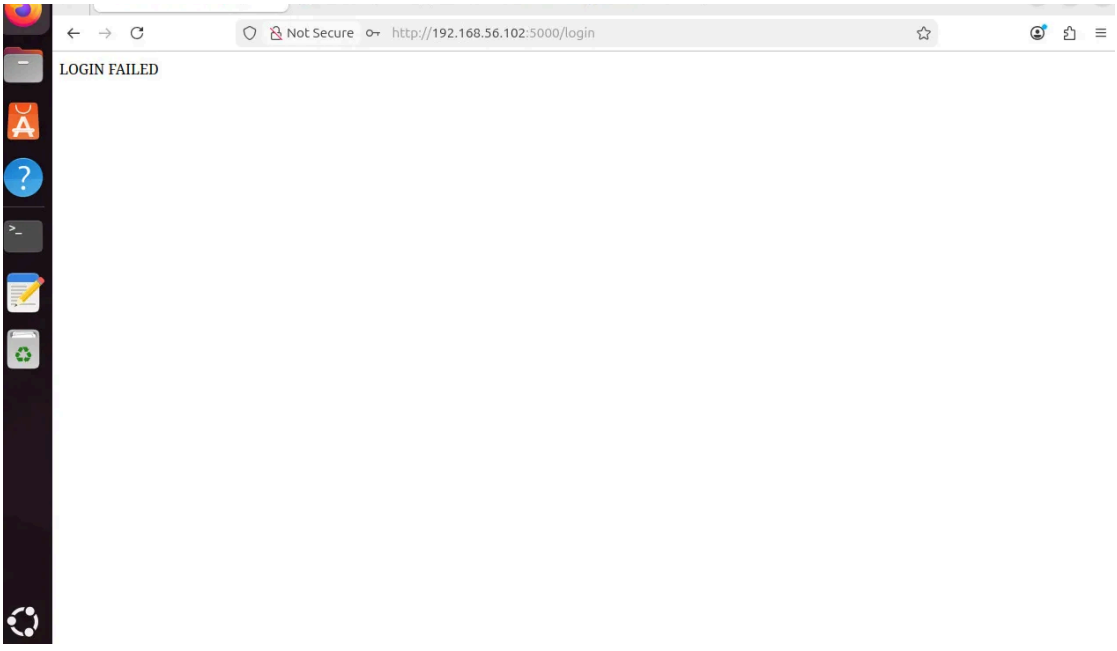
Fixed code:

```
q = "SELECT * FROM users WHERE username=? AND password=?"
r = db.execute(q, (u, p)).fetchone()
```

Let’s Try SQL Injection now:



Login is Getting Failed As You Can See:



“The SQL Injection vulnerability is fixed by using parameterized queries to ensure user input is treated as data and not executed as part of the SQL

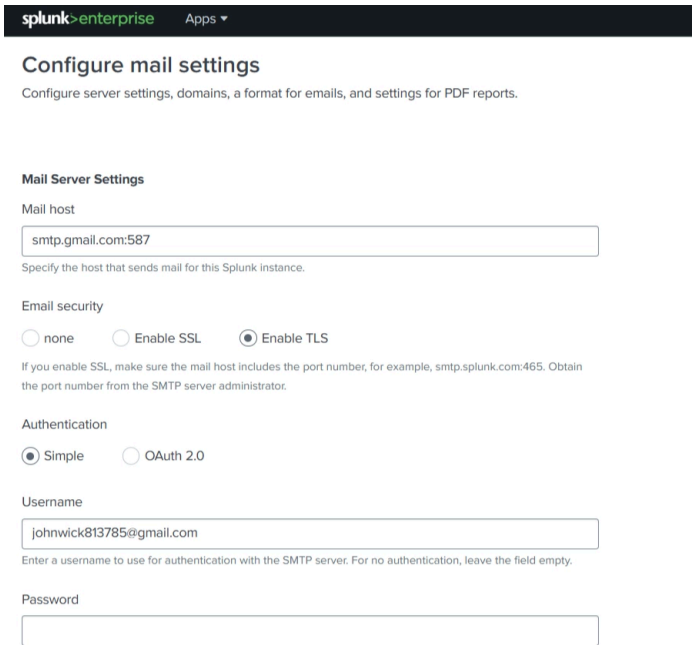
command."

ALERT CREATION AND EMAIL NOTIFICATION INTEGRATION

Now we will create a new alert and trigger real-time notifications via Email, along with a simple Telegram notification:

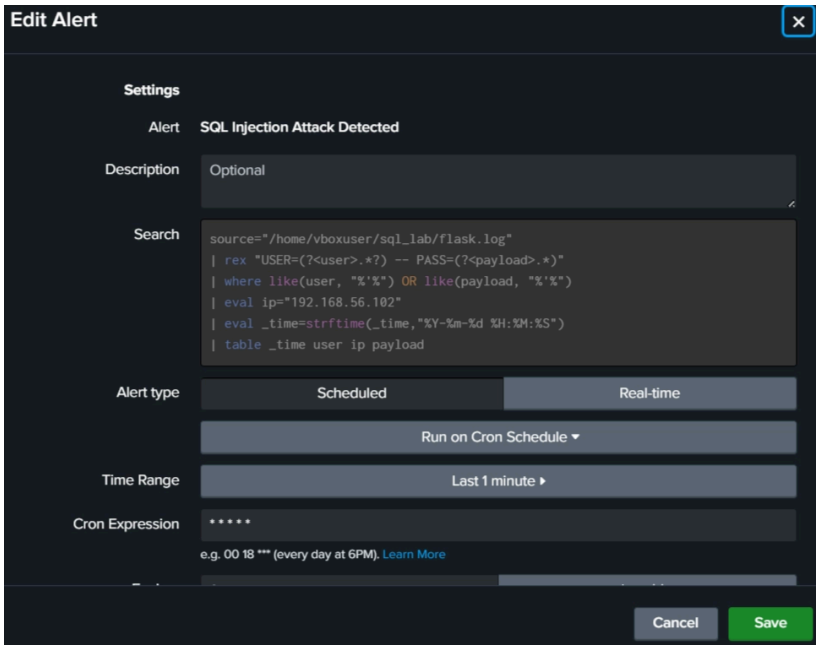
1. Configure Email Settings in Splunk

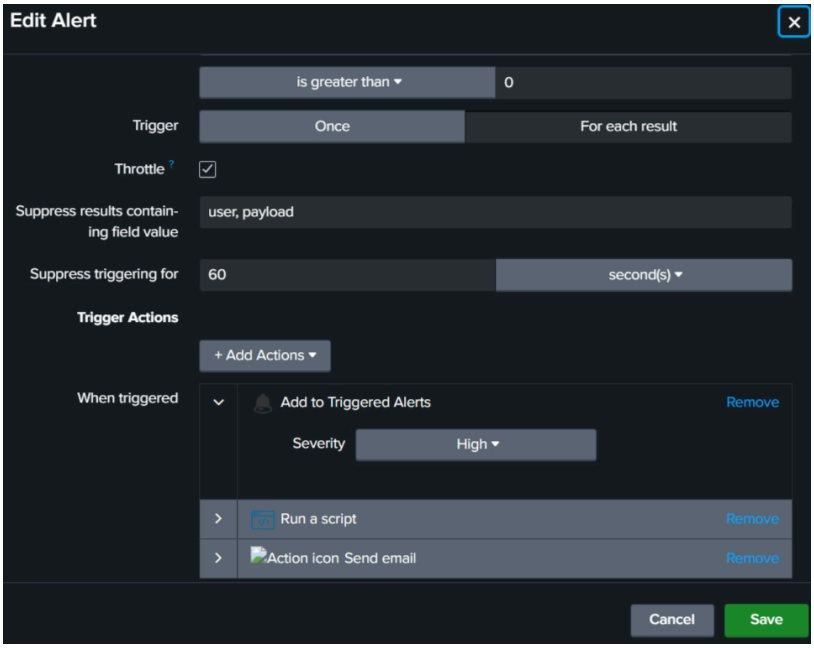
- SMTP settings were configured using Gmail to enable email notifications.
- TLS was enabled and an App Password was used for secure authentication.



2. Create a Splunk Alert

- A **new alert** was created in Splunk for SQL Injection detection.
- Set **Trigger Condition** to: Number of results > 0
- Set **Time Range** to: Last 1 minute (for real-time detection)

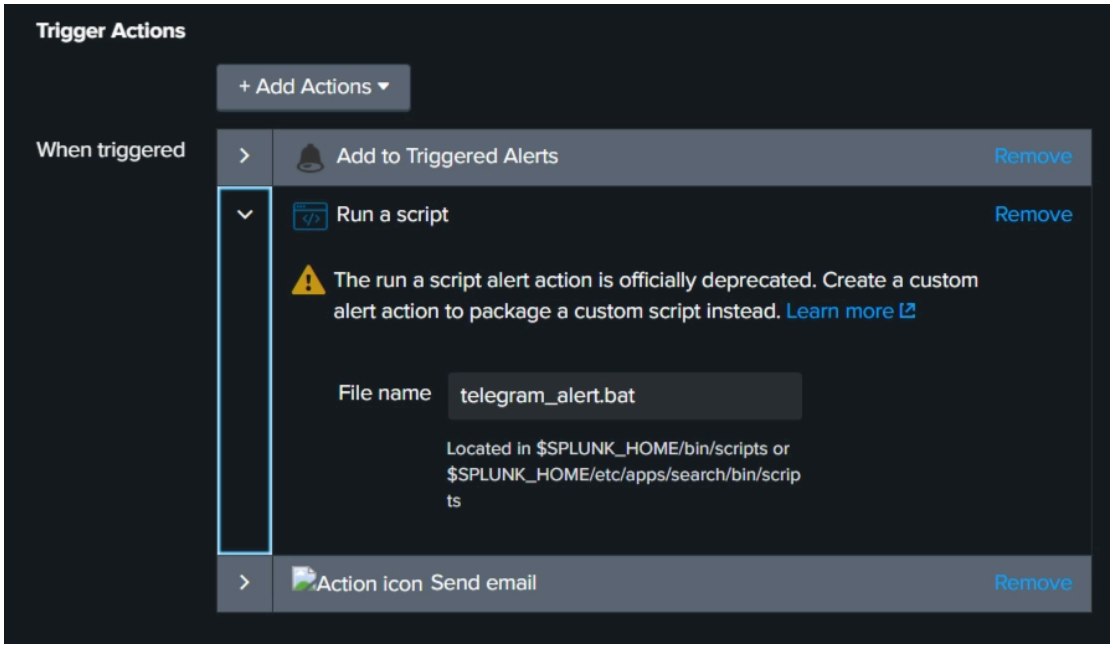


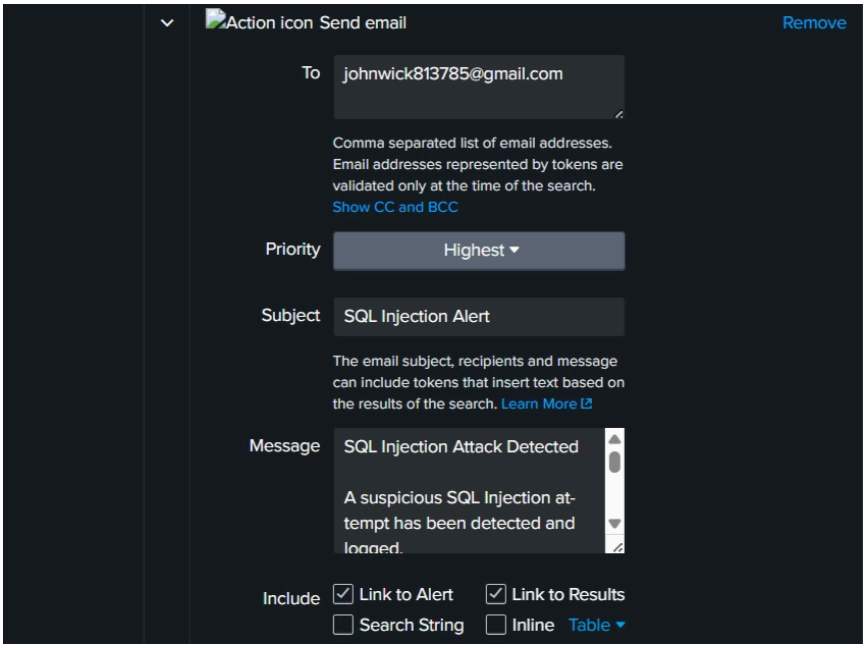


3. Configure Alert Actions

- In **Trigger Actions**, **Send Email** was enabled to receive real-time alerts.
- Additionally, a basic Telegram notification was configured using **Run a Script** with the BAT file.

- Now, whenever a SQL Injection attack occurs, Splunk triggers the alert and:
  - Sends a **real-time email notification**
  - Sends a **Telegram notification (optional)**





RESULTS

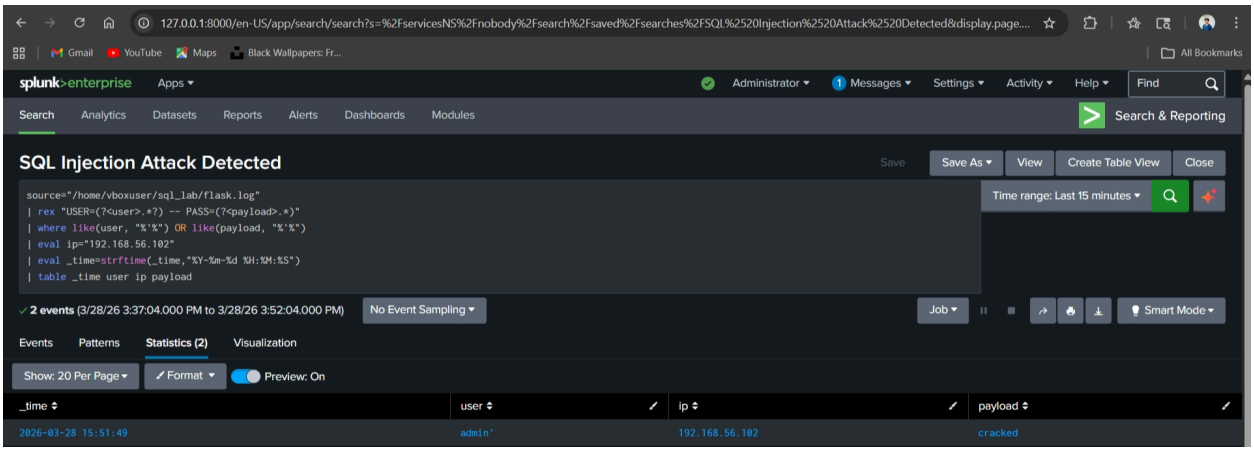


Figure 1: SQL Injection attempts detected in Splunk

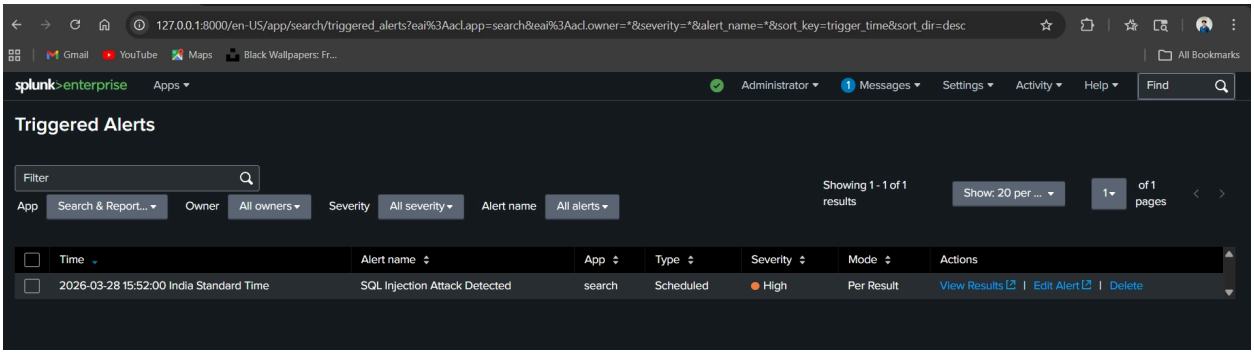


Figure 2: Alert triggered upon SQL Injection detection

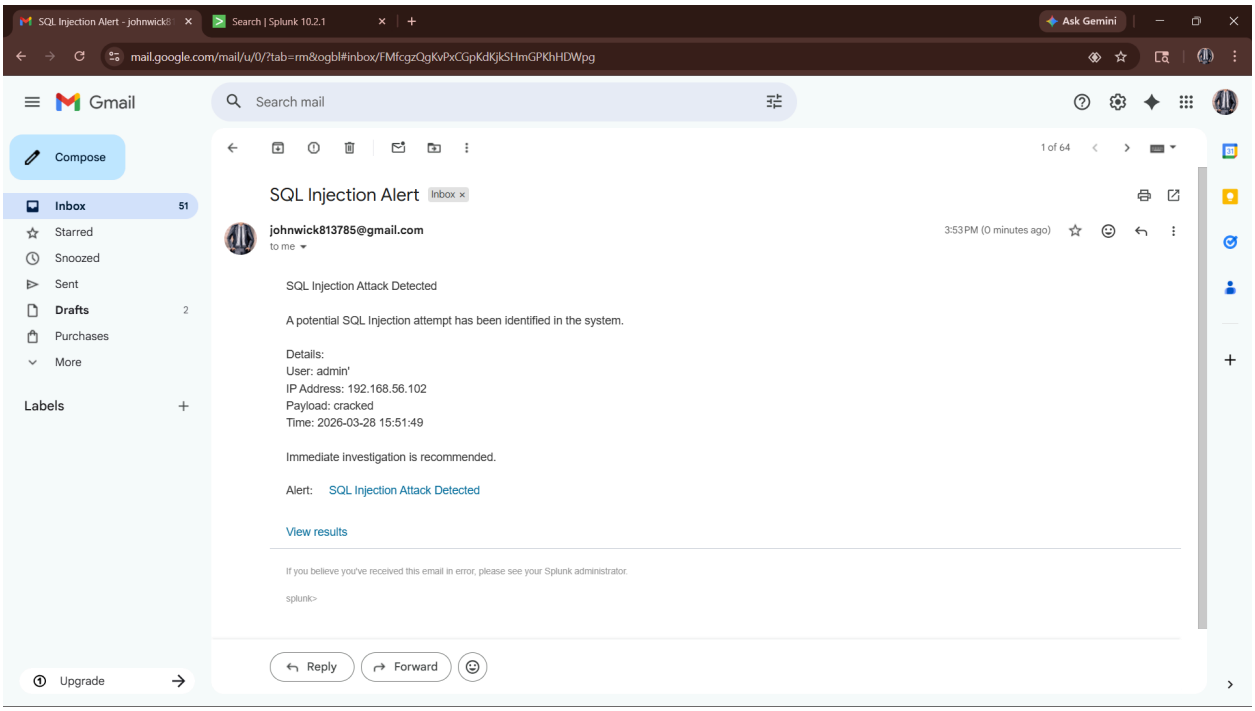


Figure 3: Real-time alert notification received via Email

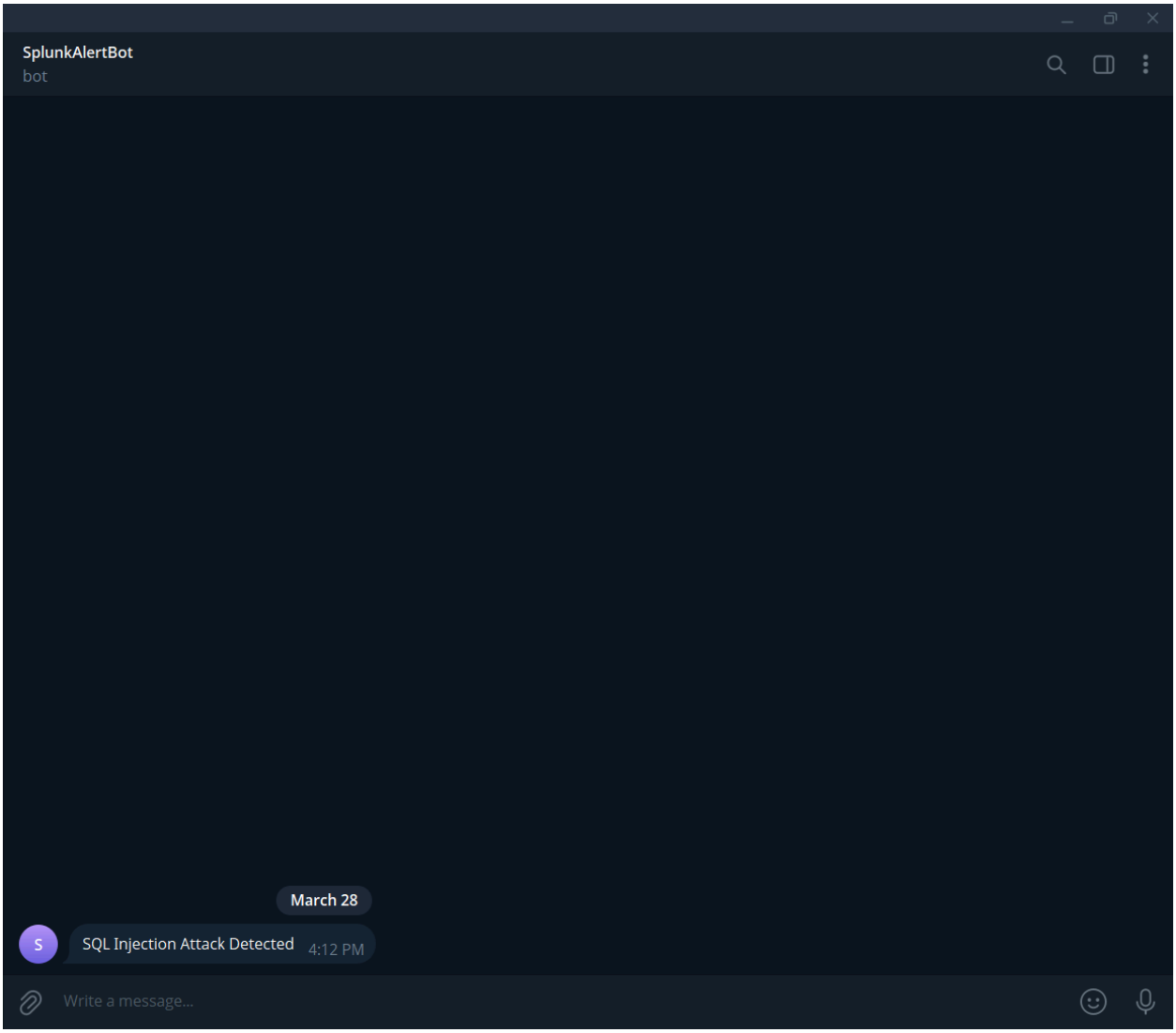


Figure 4: Real-time alert notification received via Telegram

CONCLUSION

This project successfully demonstrated the detection of SQL Injection attacks using Splunk SIEM through centralized log collection and analysis of a vulnerable web application. Real attack attempts were generated and identified by monitoring suspicious query patterns in Splunk.

In addition to detection, the SQL Injection vulnerability was mitigated by implementing parameterized queries and input validation, preventing further exploitation. The system was further enhanced by configuring alerts that were automatically triggered upon detecting malicious activity, with real-time

notifications sent via Email as the primary alert mechanism, along with a supplementary Telegram notification for enhanced visibility and immediate awareness.

Overall, this project highlights the importance of combining SIEM-based detection, real-time alerting, and secure coding practices to strengthen web application security.